

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Факультет безопасности информационных технологий

Дисциплина:

«Web Application Development»

ОТЧЕТ ПО ПРАКТИЧЕСКОМУ ЗАДАНИЮ

«LLM-Chat»

Выполнил:

Самсонов В. Г., студент группы N4151с



(подпись)

Проверил:

Чернышев В. А., преподаватель ФБИТ

(отметка о выполнении)

(подпись)

Санкт-Петербург

2026 г.

CONTENT

1	API structure	3
2	Code structure	6
3	UI	7
3.1	Authentication Screen.....	7
3.2	Active Chat Session.....	8
4	Database.....	9

1 API STRUCTURE

The backend exposes a RESTful interface augmented with Server-Sent Events (SSE) for streaming LLM responses. All protected endpoints require JWT authentication via the Authorization: Bearer <token> header. Endpoints are shown in Table 1.

Resource Group	Method	Endpoint	Description	Auth Required
Authentication	POST	/api/auth/register	Create new user (username, email, password)	No
	POST	/api/auth/login	Authenticate with credentials; returns access and refresh tokens	No
	POST	/api/auth/refresh	Exchange valid refresh token for new access/refresh token pair	No
	GET	/api/auth/github	Redirects client to GitHub OAuth authorization page	No
	GET	/api/auth/github/callback	Handles OAuth code exchange, creates/links user, redirects with tokens	No
	GET	/api/auth/me	Returns authenticated user profile	Yes

Resource Group	Method	Endpoint	Description	Auth Required
Chats	POST	/api/chats	Create new conversation thread	Yes
	GET	/api/chats	List all chats owned by the authenticated user	Yes
	GET	/api/chats/{chat_id}	Retrieve specific chat metadata	Yes
Messages	POST	/api/chats/{chat_id}/messages	Persist a user message within a chat	Yes
	GET	/api/chats/{chat_id}/messages	Retrieving chronological message history for a chat	Yes
LLM Interaction	POST	/api/chats/{chat_id}/chat/stream	Send prompt to local LLM; streams tokens via SSE	Yes

Client Usage Workflow:

- Authentication: Client submits credentials to /api/auth/login. On success, the server returns access_token (short-lived) and refresh_token (long-lived). Tokens are stored client-side.
- Protected Requests: All subsequent API calls include Authorization: Bearer <access_token>. The backend middleware validates signature and expiration statelessly.
- Token Refresh: Upon receiving 401 Unauthorized, the client calls /api/auth/refresh with the stored refresh_token. The backend verifies the token against Redis, rotates it, and issues a new pair.

— Streaming LLM: Client initiates POST /api/chats/{id}/chat/stream. The response Content-Type is text/event-stream. Each SSE payload contains JSON with { "token": "..." } until { "done": true } signals completion.

2 CODE STRUCTURE

The project strictly follows the Model–Controller–Service (MCS) architecture for an SPA backend. Responsibilities are decoupled to ensure maintainability, testability, and clear separation of concerns.

Directory structure presented in the following listing:

```
app/
├── main.py           # Application factory, middleware, startup events
├── config.py         # Pydantic Settings, environment variable parsing
├── database.py       # Async SQLAlchemy engine, session factory
├── security.py       # JWT generation/verification, password hashing
├── dependencies.py   # FastAPI dependency injection (user extraction)
├── models/          # ORM definitions (Data Layer)
├── routers/         # FastAPI route handlers (Controller Layer)
├── services/        # Business logic, transactions, external integrations
└── frontend/        # SPA assets (HTML, CSS, Vanilla JS)
```

Layer responsibilities presented in Table 2.

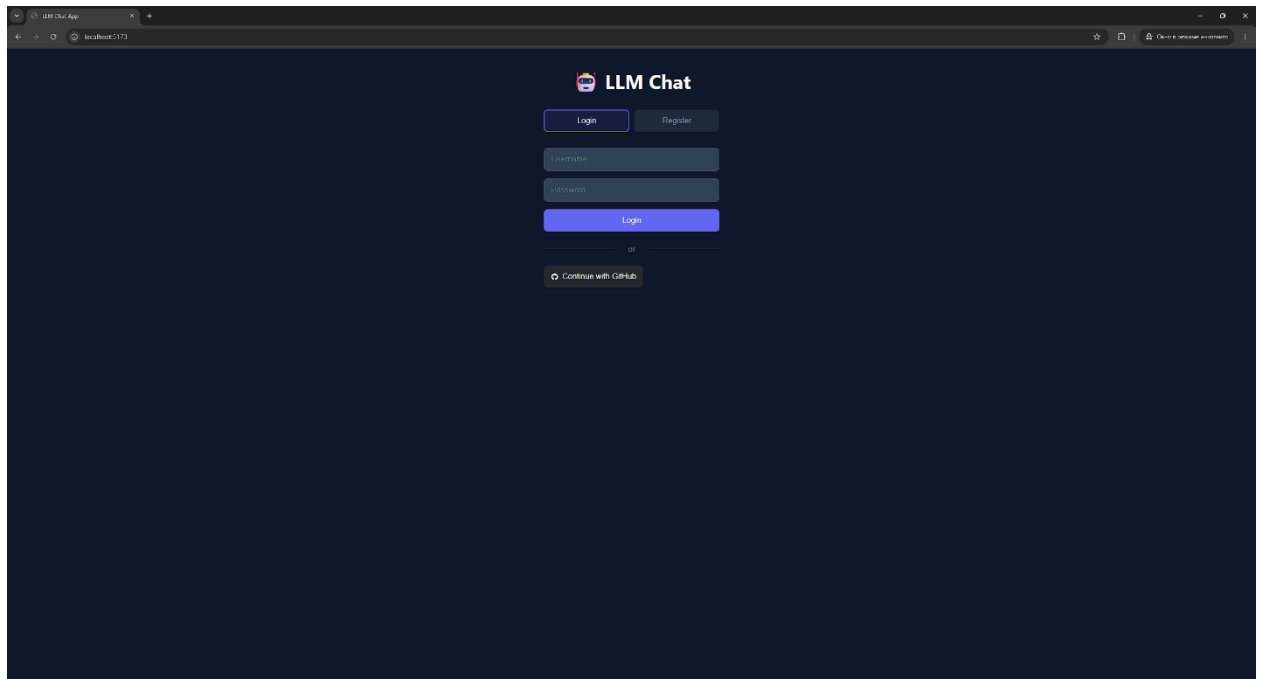
Layer	Directory/Module	Responsibility
Model	app/models/	SQLAlchemy ORM classes mapping to PostgreSQL tables. Define table schemas, foreign keys, and relationship mappings. Stateless data representations.
Controller	app/routers/	FastAPI routers acting as HTTP controllers. Handle request routing, Pydantic schema validation, dependency injection (Depends(get_current_user)), and response serialization. Contain zero business logic.
Service	app/services/	Encapsulates all operational logic: database transaction boundaries, JWT/Redis session management, GitHub OAuth token exchange, and LLM inference orchestration via llama-cpp-python.
Infrastructure	config.py, database.py, security.py	Cross-cutting concerns: environment configuration, async database connection pooling, cryptographic utilities, and JWT lifecycle management.

3 UI

The frontend is implemented as a lightweight Single-Page Application (SPA) using vanilla JavaScript, HTML5, and CSS3. It communicates with the backend via the fetch API and ReadableStream for SSE consumption.

3.1 Authentication Screen

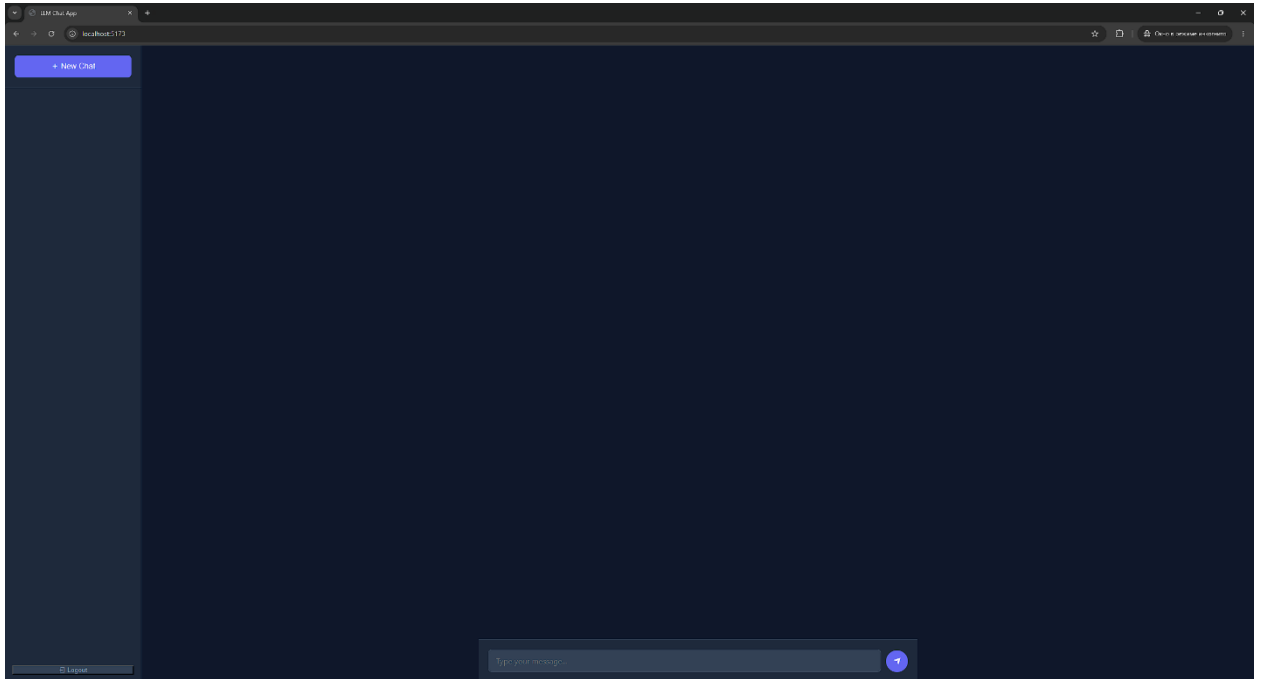
Login and registration forms with credential validation. Contains a dedicated GitHub OAuth button that redirects to the GitHub authorization page.



Picture 1 – Authentication Screen

3.2 Active Chat Session

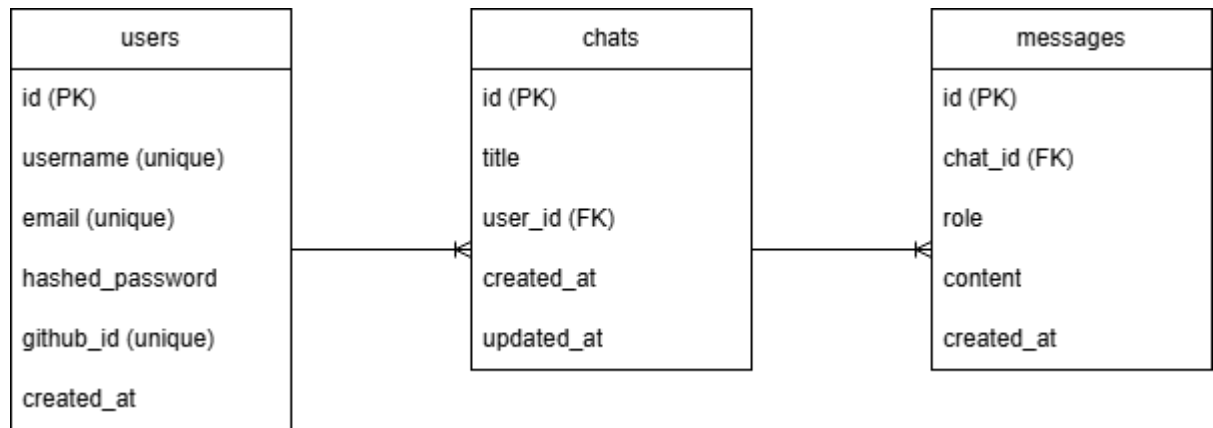
Main chat area displaying chronological message history. Shows real-time token streaming during LLM inference with automatic scrolling. Left navigation panel displaying a list of persistent conversation threads. Includes a "New Chat" button and logout functionality.



Picture 2 – Main Area

4 DATABASE

The application uses PostgreSQL as the primary relational database. Schema evolution is managed via Alembic migrations. All tables utilize `TIMESTAMP WITH TIME ZONE` and enforce referential integrity through foreign key constraints.



Picture 3 – Entity Relationship Description